

Claude e Claude Code in pratica

Dal prompting agli agenti

Simone Scannapieco

Corso base per Tecnodata Trentina S.r.l., Trento, Italia

Modulo 2 — Misurare e migliorare

Il prompt come programma

Il primo prompt è una bozza

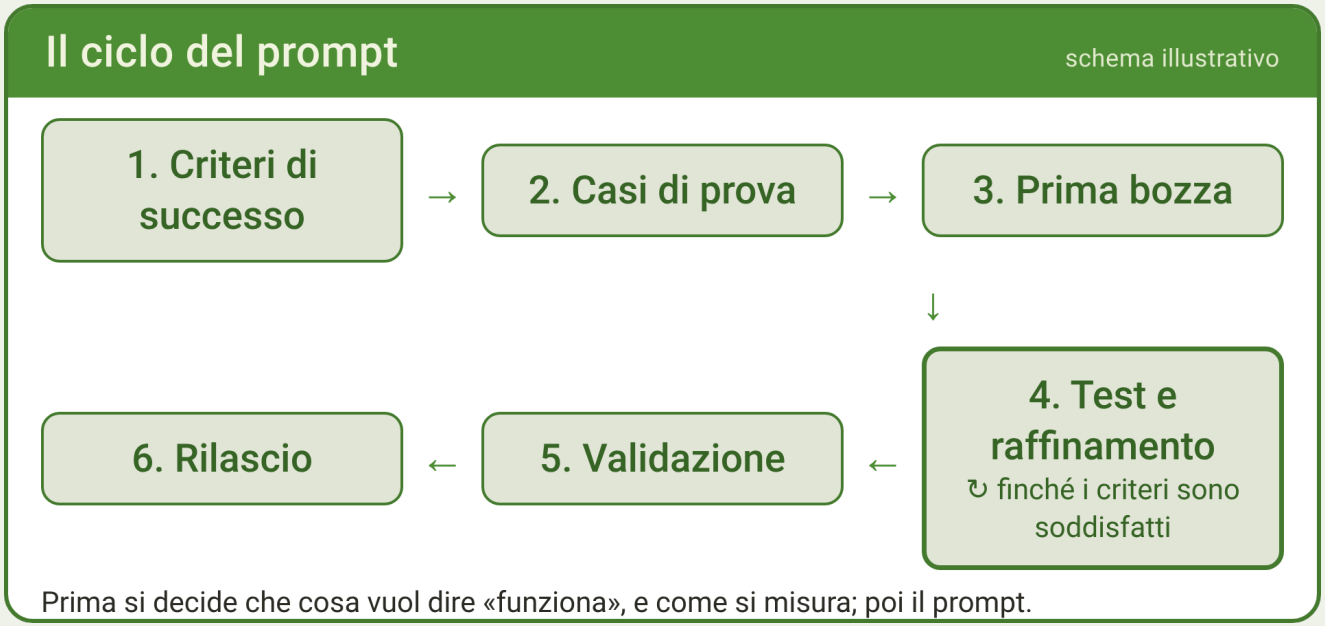
Un prompt che usate davvero si scrive, si prova e si migliora come un programma

- La risposta ai clienti, la spiegazione di una funzione, la lettura di un log: un prompt che usate ogni giorno è **un programma**, scritto in linguaggio naturale — e la sua prima versione è una bozza
- **Una variabile alla volta**: il testo, oppure il modello, oppure l'effort — se ne cambiate due insieme, non sapete quale ha fatto la differenza. E, consiglia Anthropic, un gruppo di esempi **tenuto da parte**, per scoprire quando il prompt si adatta troppo ai casi di prova
- **Ogni versione in un file**, `v0.md`, `v1.md`, …, con una riga di registro — che cosa avete cambiato, che cosa è successo — e `git diff` per vedere la differenza. Ogni modifica al prompt, scrive Anthropic, **si misura prima di rilasciarla**
- Se correggere non basta, **non insistete**: dopo due correzioni fallite la guida di Claude Code consiglia `/clear` e un **prompt migliore**, invece di una terza correzione
- **Sperimentare insieme**: un collega che legge il vostro prompt senza il vostro contesto vede ciò che voi non vedete più

> *No man is an island, entire of itself; every man is a piece of the continent, a part of the main. — Nessun uomo è un'isola, completo in sé stesso; ogni uomo è un pezzo del continente, una parte della terraferma.*
>
> **John Donne, «Devotions upon Emergent Occasions», Meditazione XVII, 1624; la traduzione è del corso**

Il ciclo di Anthropic

Prima i criteri e il modo di misurarli, poi il prompt



Il ciclo del prompt come lo descrive la documentazione di Anthropic: dai criteri di successo al rilascio
Test e raffinamento si ripetono, finché i criteri sono soddisfatti
© Simone Scannapieco — schema illustrativo, disegnato per questo corso

- Prima del prompt, i **criteri di successo** — che cosa vuol dire «funziona» — e le valutazioni che li **misurano**: per Anthropic è un ciclo centrale nel prompt engineering
- Poi i **casi di prova**, casi limite compresi, e una **prima bozza**; test e raffinamento **si ripetono**; alla fine la **validazione**, e il rilascio
- La Claude Academy insegna lo stesso ciclo in cinque passi: un obiettivo, un prompt iniziale, la valutazione, le tecniche, **una nuova valutazione**

Come si misura

Tre modi, dalla documentazione di Anthropic — e il giudice del laboratorio

- **A voce**: come decidete, nel vostro lavoro, che un prompt è migliore di un altro?
- **Controlli scritti nel codice**: i più veloci e affidabili, ma senza sfumature — il telefono che non deve esserci, la lunghezza, un campo che manca
- **Un giudice**: un modello che legge una **rubrica** dettagliata e chiara e dà un esito netto, giusto o sbagliato, o da 1 a 5; veloce, flessibile, adatto ai giudizi complessi — meglio se **ragiona** prima di decidere. E prima di fidarsene, scrive Anthropic: «**Prima verifica** che sia affidabile, poi scala»
- **La vostra lettura**: la più flessibile, ma lenta e costosa — per ciò che il codice e il giudice non decidono
- Dal blog degli sviluppatori di Anthropic: la rubrica come **affermazioni verificabili**, ciascuna rispettata o no — non una scala da 1 a 5 — e un modello giudice **diverso** da quello che valutate
- Un revisore **indipendente**, senza il ragionamento che ha prodotto il lavoro, coglie i problemi sottili **meglio dell'autorevisione**: in Claude Code, `/code-review` rivede il diff in un subagent **nuovo**, che non ha visto la conversazione
- Nel **laboratorio** di questa seconda parte il giudice è un **subagent**, un file con il suo prompt e il suo modello: per ogni affermazione scrive **prima il motivo, poi l'esito** — **Haiku** giudica le risposte di **Sonnet**; sulla prima versione lo si chiama **due volte**, come fa Anthropic, e un'affermazione su cui i due giudizi non concordano è «instabile»

I limiti del giudice

Anche chi misura sbaglia: in che modo, e che cosa si fa

- Un revisore a cui chiedete di **trovare problemi** ne troverà **sempre** qualcuno, e correggendo ogni segnalazione si finisce con astrazioni in più e test per casi che non possono capitare. La guida di Claude Code, nel suo esempio: chiedetegli le lacune che toccano la **correttezza**, e di **lasciar perdere lo stile**
- Uno studio del 2023 sui giudici LLM (Zheng e altri, NeurIPS) descrive due tendenze: preferire **una posizione**, la prima o la seconda risposta, e preferire le risposte **più lunghe**, anche quando non sono migliori
- Il giudice è **uno strumento**, non l'arbitro — per la Claude Academy i giudici modello «possono essere un po' **capricciosi**», ma danno una base costante per misurare i miglioramenti (traduzione nostra). Leggete i motivi, e quando non siete d'accordo **decidete voi**; se serve, riscrivete l'affermazione della rubrica perché si verifichi senza ambiguità

Fatevi fare le domande

Prima di scrivere il prompt, e dentro il prompt

- Prima di scriverlo, chiedete a Claude di **farvi domande** sul compito, **poi** di scrivere il prompt: lo suggerisce un tutorial della Claude Academy, e Anthropic lo chiama *interview pattern* — le domande fanno emergere ciò che non avevate previsto

claude.ai o Claude Code, prima di scrivere il prompt

```
Devo scrivere un prompt per rispondere ai clienti che
segnalano un guasto. Prima di scriverlo, fammi le domande
che ti servono sul compito: chi legge la risposta, che cosa
possiamo promettere, che cosa fare quando manca un dato.
Poi scrivi il prompt.
```

- E nel prompt, il **permesso di dire «non lo so»**: una tecnica che, scrive Anthropic, può ridurre **drasticamente** le informazioni false

La checklist del prompt, in otto punti

Da scorrere prima di premere Invio — e di nuovo quando la risposta non convince

1. **Ruolo**: chi è Claude, per chi lavora
2. **Contesto**: lo scopo, chi legge, i dati e le regole che valgono
3. **Una richiesta sola** per prompt: un compito circoscritto dà risultati migliori di tanti obiettivi insieme
4. **Formato**, o uno schema se il risultato lo legge un programma
5. **Vincoli in positivo**: che cosa fare, e che cosa scrivere quando un dato manca; il permesso di dire «non lo so»
6. **Esempi**, se la forma conta, con le etichette ammesse
7. **Ogni ingrediente e ogni dato nel suo tag**, e nessuna memoria presunta: ogni chiamata parte da ciò che le date
8. **Il test del collega**: lo capirebbe chi non sa niente del compito?

Il prompt engineering è debugging del linguaggio, non un incantesimo.

La checklist è nella scheda `riferimenti/pattern-di-prompt.md` del repository delle esercitazioni